

Optimizing Large-Scale Song Search Using Hash Tables

Student Name: Tulsi Mansukhbhai Hudka

Course: CS 5329 - Algorithm Design and Analysis

1. Introduction

For this project, I stayed with the same idea I used in my proposal and discussion post: fast song lookup in a Spotify-style dataset. The main question I wanted to answer was simple: when a system has to search a large music library repeatedly, how much difference does the data structure make?

To test that, I compared two approaches:

- a **baseline linear scan**, which checks every record one by one
- an **optimized hash table lookup**, which builds an index first and then answers queries directly

I hypothesized that the linear scan would be simpler and use less extra memory, but the hash table would be much faster once the same dataset had to support repeated searches.

2. Problem Definition

The input to the project is a CSV file containing song records. Each row includes a title and related metadata such as artist, album, duration, and popularity. The task is to return all records whose title matches the user's search query.

This problem is a good fit for algorithm analysis because correctness is easy to verify, while the performance gap between approaches becomes much more obvious as the dataset size grows.

3. Algorithms Compared

3.1 Baseline: Linear Scan

The baseline algorithm loops through the full dataset and compares the query against every song title. If the title matches, the record is added to the results.

- **Time complexity:** $O(n)$ per query
- **Space complexity:** $O(k)$ for returned matches

This approach is easy to implement and does not require any preprocessing, but every new search repeats the same full pass over the dataset.

3.2 Optimized: Hash Table

The optimized algorithm first builds a hash table where:

- the **key** is the normalized song title

- the **value** is a list of matching records

After that preprocessing step, the system can answer a query through direct lookup instead of scanning the whole file again.

- **Build time:** $O(n)$
- **Lookup time:** $O(1)$ average
- **Space complexity:** $O(n)$

This approach uses more memory, but it is much better when many searches are performed on the same dataset.

4. Experimental Setup

To keep the comparison fair, both algorithms were tested on the same synthetic Spotify-style datasets with **10,000**, **50,000**, and **100,000** records. I used a synthetic data generator so the project can run immediately without depending on an external download. The generated files include repeated popular titles along with many unique titles.

The benchmark script measures:

- total runtime using `time.perf_counter()`
- memory usage using `tracemalloc`
- correctness by comparing optimized results against baseline results

For each dataset size, I ran **200 queries**. The query set included:

- common song titles that appear many times
- unique titles from the dataset
- a title that does not exist in the dataset

That setup matters because the hash table is most useful when the same indexed dataset is searched repeatedly.

5. Results

5.1 Runtime Results

Dataset Size	Linear Scan Total	Hash Build Time	Hash Search Time	Hash Total
10,000	0.3301 s	0.0065 s	0.000053 s	0.0066 s
50,000	1.6765 s	0.0220 s	0.000054 s	0.0220 s
100,000	3.3457 s	0.0405 s	0.000052 s	0.0405 s

5.2 Memory Results

Dataset Size	Linear Peak Memory	Hash Peak Memory
10,000	61,294 bytes	1,555,610 bytes
50,000	62,639 bytes	9,043,363 bytes
100,000	63,682 bytes	18,087,378 bytes

5.3 Correctness

For all tested dataset sizes, the optimized method returned the same results as the linear scan, so the correctness check passed in every case.

6. Analysis

The results supported the overall hypothesis.

First, the linear scan scaled poorly. As the number of records increased, total runtime increased sharply because each query still required checking the dataset from beginning to end. That matches the expected $O(n)$ behavior.

Second, the hash table had a clear upfront indexing cost and much higher memory usage. That is the main trade-off in this project. The optimized approach is not free: it pays for speed with extra preprocessing and storage.

Even with that cost included, the indexed approach was still much faster overall for repeated searches. At 100,000 records, the total time for 200 linear scans was **3.3457 seconds**, while the full indexed approach, including building the hash table and running all queries, took only **0.0405 seconds**. The search-only portion of the indexed method was even smaller, around **0.000052 seconds**.

That difference matters in practice. If a system only had to answer one query, a linear scan might still be acceptable for moderate input sizes. But for a music application where users repeatedly search in the same library, preprocessing the dataset into a hash table is a much more practical solution.

7. Conclusion

This project shows that algorithmic choice has a direct impact on performance in large-scale search tasks. The linear scan is simple and memory-efficient, but it becomes much slower as repeated searches are performed on larger datasets. The hash table uses more memory and requires preprocessing, but it dramatically reduces lookup time and scales far better for repeated-search workloads.

Based on both the theoretical analysis and the measured results, the hash table is the better choice for a Spotify-style song lookup system when fast repeated searches are important.

8. Future Improvements

If I extended this project further, I would consider:

- adding prefix or partial-title search instead of exact-title matching

- indexing by both title and artist
- benchmarking on a real Kaggle Spotify dataset in addition to synthetic data
- adding a separate chart for memory growth